

oxide-workflow

Practical guide: configuring runs and driving them from the terminal

The pipeline relaxes a four-stage chain with a machine-learned interatomic potential (MLIP): **bulk** → **slab** → **oxygen vacancy** → **adsorbate**. Each stage takes the previous stage's relaxed geometry, applies one modification, and relaxes again. The vacancy and adsorbate stages are *screenings*: every symmetry-distinct site is relaxed and the lowest-energy one is carried forward.

The headline output is **E_{ads}**, the adsorption energy, computed as $E(\text{substrate+adsorbate}) - E(\text{bare substrate}) - E(\text{isolated fragment})$, with all three terms from the same calculator.

The one thing to understand first

Every number in `oxide_workflow/config.py` has a matching command-line flag. You can change a setting either by editing that file (it becomes the new default for everything) or by passing the flag (it applies to that run only). The flag always wins. Nothing else needs editing to change a run.

Where to configure

Slab geometry — SlabConfig

Controls what surface gets cut and how much of it is allowed to move.

Setting in config.py	Default	Flag	What it does
<code>miller_index</code>	(1,1,0)	<code>--miller</code>	Which facet to cut. Pass as "110", or "1,1,-1" with commas for negatives.
<code>min_slab_size</code>	12.0	<code>--thickness</code>	Minimum slab thickness in angstroms. This is the layer-count knob: 12 → 20 takes rutile(110) from 24 to 42 atoms per 1x1 cell.
<code>min_vacuum_size</code>	12.0	<code>--vacuum</code>	Vacuum gap above the surface, in angstroms. Stops the slab seeing its own periodic image.
<code>termination_index</code>	1	<code>--termination</code>	Which termination of that facet. An index into pymatgen's list, not a literature quantity — see the warning below.
<code>supercell</code>	(4,2)	<code>--supercell</code>	Lateral replication. The dominant cost knob : 4,2 is 192 atoms, 1,1 is 24. Also sets defect and adsorbate coverage.
<code>freeze_bottom_fraction</code>	0.5	<code>--freeze</code>	Fraction of slab thickness held at bulk positions. 0.8 freezes the bottom 80%, leaving a thinner active surface.
<code>max_vacancy_sites</code>	None	<code>--max-vacancy-sites</code>	Cap the vacancy screening. Smoke tests only — keeps the first N by site index, so it can drop the true minimum.

termination_index is an index, not a literature value. Literature names a physical surface ("the stoichiometric bridging-oxygen (110)"); which integer that lands on is pymatgen's ordering and can differ between materials. Index 1 has been verified correct for all seven rutiles in this project. If you add a new material, run `validate_materials.py` first — it reports what each index actually is, and a wrong pick fails silently with plausible-looking numbers.

Relaxation — RelaxConfig

Setting	Default	Flag	What it does
<code>fmax</code>	0.02	<code>--fmax</code>	Force convergence target in eV/angstrom. The biggest speed lever after supercell . Loosen to 0.1-0.2 for smoke tests.
<code>max_steps</code>	300	<code>--max-steps</code>	Optimizer step cap per relaxation. Most relaxations converge near 90 steps.
<code>optimizer</code>	FIRE	(edit file)	FIRE or BFGS.

Adsorbate — AdsorbateConfig

Setting	Default	Flag	What it does
species / coords	H	--adsorbate	Which fragment. Pick by name from the table below rather than editing coordinates.
positions	ontop , bridge, hollow	(edit file)	Which site types the geometric site-finder enumerates.
max_per_position	None	--max-sites	Sites kept per position type. See "Why cap the sites" below — this matters scientifically, not just for cost.
seed_standoff	0	(edit file)	Extra angstroms added to the covalent bond length when placing the fragment.
min_clearance	0.8	(edit file)	Reject a placement closer than this multiple of the covalent bond length. Guards against spawning inside a surface atom.
surface_depth	3.0	(edit file)	How far below the top atom to scan for exposed surface atoms.

Models and material — RunConfig

Setting	Default	Flag	What it does
polymorph	rutile-tio2	--material	mp-id, built-in alias, or a path to your own CIF/POSCAR.
reference	MACE-mh1-omat	(edit file)	The ground-truth model everything is compared against.
candidate	MACE-mh1-oc20	--candidates	Model(s) benchmarked against the reference. The flag accepts several.

Adsorbate fragments

Name	Atoms	When to use it
H	1	Hydrogen. Hydroxyl formation on bridging oxygen.
CO	2	Carbon monoxide, C-down. The classic probe molecule.
H ₂ O	3	Water, O-down.
O ₂	2	Oxygen, end-on. The physically correct oxygen reservoir — atomic O does not free-float.
O ₂ -side	2	Oxygen lying flat. The better start when you expect dissociation at a vacancy; an end-on start rarely finds it.
O	1	Atomic oxygen. Direct vacancy-healing probe, but the shakiest gas reference (see below).

Two caveats on oxygen. O₂'s gas reference inherits the well-known GGA overbinding error of a few tenths of an eV, which every PBE-trained model here shares. Atomic O is worse: its reference is an isolated atom, and no model here carries a spin state. In both cases the error is a *constant* within one model, so site rankings and E_{ads} differences are unaffected — but do not read an absolute E_{ads} as physical truth, and be careful comparing absolute values between models.

Registered models

Two multi-head checkpoints, each head registered separately. Any can be reference or candidate. Each runs in its own conda environment, launched as a subprocess — the orchestrator never imports a model, which is how mutually incompatible environments coexist.

Model	Training labels	Model	Training labels
MACE-mh1-omat	OMat24, PBE (default reference)	UMA-oc22	OC22, oxide catalysis
MACE-mh1-oc20	OC20, PBE-surf (default candidate)	UMA-oc20	OC20, catalysis
MACE-mh1-mp	MPtrj, PBE	UMA-oc25	OC25
MACE-mh1-matpes	MatPES, r2SCAN	UMA-omat	OMat24, bulk
MACE-mh1-spice	SPICE, wB97M	UMA-odac	ODAC, MOF
MACE-mh1-omol	OMol, molecular	UMA-omc / UMA-omol	molecular crystals / molecules

The scripts

Script	Use it for	Needs API key
launch.py	Interactive menus. Every option is read from the live registries, so it doubles as the discovery tool. Prints the equivalent command before running.	no
run_stage.py	Run one stage only , on any starting structure. The play-around tool — change one knob, see the effect in minutes.	no
run_family.py	The whole rutile family in one command. Restartable.	no
validate_materials.py	Dry structural check with no relaxation. Run this before spending CPU on a new material.	no
report.py	Summarise a finished or in-progress run directory.	no
fetch_rutiles.py	Download the rutile family from Materials Project. One-time.	yes
fetch_structure.py	Download one material by mp-id. One-time.	yes

The API key: when and where

Only the two `fetch_*` scripts ever need network access or a key. The pipeline itself never reads it: structures are saved to `data/structures/` once and resolved offline forever after (which is also what makes runs reproducible, and what will let this work on compute nodes with no outbound network). Get a key from materialsproject.org under Profile → API, then set it in your shell — never in a file, since these are committed.

```
export MP_API_KEY=your_key_here
python scripts/fetch_rutiles.py
```

To use a structure that Materials Project does not have, skip the key entirely and point `--material` at your own file:

```
python scripts/run_stage.py slab --material /path/to/my_structure.cif
```

Use cases

Every command below has been run and verified. All of them accept the full set of geometry flags from the configuration tables, so treat these as templates.

1. Quick bulk relaxation

The fastest possible check that a material and model work together. Seconds.

```
python scripts/run_stage.py bulk
# a different material and model
python scripts/run_stage.py bulk --material mp-825 --model MACE-mh1-oc20
```

Writes `runs/stage/<model>/bulk/` with POSCAR, CONTCAR, trajectory.xyz, OUTCAR.

2. Quick slab relaxation

Relaxes the bulk first, then cuts and relaxes the slab. Skips vacancy and adsorbate, which are the expensive stages.

```
python scripts/run_stage.py slab
# small and loose, for a fast look
python scripts/run_stage.py slab --supercell 1,1 --fmax 0.2 --max-steps 20
```

3. Thicker slab with 80% frozen instead of 50%

Both knobs at once. Thickness is in angstroms, not layers — the number of atomic layers follows from the material's lattice constant.

```
python scripts/run_stage.py slab --thickness 20 --freeze 0.8
```

On rutile(110) at 1x1 this goes from 24 atoms with 12 frozen (50%) to 42 atoms with 34 frozen (81%). The printed header reports both, so you can confirm the change took effect before the relaxation starts.

4. Change the facet

Miller indices as a bare string, or comma-separated when any index is negative.

```
python scripts/run_stage.py slab --miller 101
python scripts/run_stage.py slab --miller 1,1,-1
```

A new facet usually has a different number of terminations. Check with `validate_materials.py --miller 101` before trusting `termination_index`.

5. Change the termination

Which of the facet's possible surface cuts to use.

```
# see what terminations exist and what each one is
python scripts/validate_materials.py mp-2657 --miller 110

# then pick one
python scripts/run_stage.py slab --termination 0
```

The validator reports stoichiometry, polarity and symmetry per termination. A polar or off-stoichiometry slab has a meaningless surface energy, so this is worth checking rather than assuming.

6. Vacancy screening on an already-relaxed slab

This is the big time-saver. --from takes any CONTCAR/POSCAR/CIF and treats it as the previous stage's relaxed output, skipping everything before it. Reuse a slab you already paid for instead of re-relaxing bulk and slab.

```
python scripts/run_stage.py vacancy \
    --from runs/stage/MACE-mh1-omat/slab/CONTCAR
```

Relaxes every symmetry-distinct oxygen vacancy, ranks them, and writes rankings.csv. Match --freeze to whatever the slab was built with, or the frozen region will not line up.

7. Adsorbate screening on an already-relaxed vacancy slab

Same idea one stage further on. Places the fragment at every enumerated site, relaxes each, reports the minimum E_ads.

```
python scripts/run_stage.py adsorbate \
    --from runs/stage/MACE-mh1-omat/vacancy/site37_2e/CONTCAR \
    --adsorbate O2 --max-sites 6
```

Because the geometry came off disk with no energy attached, this first relaxes the bare substrate with the same model — that is the E_ads reference state, and borrowing another model's energy would leave a tens-of-eV offset uncanceled.

8. Full benchmark run, one material

The complete chain, reference model versus one or more candidates.

```
python -m oxide_workflow.pipeline --material mp-825 \
    --adsorbate O2 --max-sites 6 --protocol full_pipeline \
    --candidates MACE-mh1-oc20 --outdir runs/ruo2_o2
```

Or drive the same thing from menus with: python scripts/launch.py

9. The whole rutile family

Seven materials in one command. Restartable: rerun the identical command and it skips completed materials and retries only failures.

```
python scripts/run_family.py --dry-run      # plan and cost, runs nothing
python scripts/run_family.py                # go
```

10. A fast smoke test before committing real CPU

Shrinking the supercell is what makes this quick — cost scales with atom count per step, so 1,1 is roughly eight times cheaper per step than 4,2.

```
python scripts/run_family.py --outdir runs/smoke \  
  --supercell 1,1 --cap 1 --max-vacancy-sites 2 \  
  --fmax 0.2 --max-steps 20
```

Checks that all seven materials run end to end in about 20 minutes. The numbers are not usable — the cell is too small and the vacancy cap can drop the true minimum — but that is not what it is for.

Two choices worth understanding

Protocols: --protocol

Value	What it does	When
full_pipeline	Each stage built from the candidate's <i>own</i> relaxed previous stage. Realistic accumulated error.	Default. What you get screening with the model alone.
seeded	Each stage built from the <i>reference's</i> relaxed previous stage. Intrinsic per-stage error.	Stage-by-stage comparison against a ground truth such as DFT.
both	Runs each.	Roughly double the candidate cost; the reference chain is shared.

Why cap the adsorbate sites

--max-sites looks like a cost knob but is also a correctness one. Uncapped, the site-finder returns a different number of sites per material for reasons that are not chemical: identical 192-atom slabs with identical 40-atom exposed surfaces give 63 sites for TiO_2 but 116 for RuO_2 , purely because pymatgen's symmetry reduction merges fewer near-equivalent sites on some substrates.

Left alone, that gives some materials a denser search than others, so they find lower minima for reasons unrelated to their chemistry. A fixed cap gives every material the same sampling density, which is what makes E_{ads} comparable across a family.

The cap is not blind truncation. Sites are seeded one per distinct surface environment, most coordinatively unsaturated first, and only then filled out by farthest-point spread. On rutile(110) there are four distinct environments, so --max-sites 4 covers all of them and 6 leaves margin. At a cap of 2 what gets dropped is the saturated Ti6c and O3c, keeping the reactive Ti5c and bridging oxygen.

Reading the output

```
runs/<name>/
  summary.json           headline numbers, min  $E_{\text{ads}}$ 
  divergence.jsonl       candidate vs reference, per stage
  candidates.jsonl       every site's energy and  $E_{\text{ads}}$ 
  <model>/
    bulk/ slab/          POSCAR CONTCAR trajectory.xyz OUTCAR
    vacancy/
      header.json         canonical pick, margin to runner-up
      rankings.csv        every site, ordered, best first
      site37_2e/          one folder per relaxed site
    adsorbate/ ... same shape, plus  $E_{\text{ads}}$  per site
```

rankings.csv is usually what you want: rank, folder, site index, symmetry class, energy, dE from minimum, E_{ads} , convergence and quality flags. The folder column points straight at the geometry that produced each row.

```
python scripts/report.py runs/ruo2_o2
```

Check the margin before believing a site ranking. header.json records margin_to_runner_up_eV. In one real run the winning adsorbate site beat second place by 1.3 meV, with the top five spanning 15 meV — far below any MLIP's accuracy. Those sites are degenerate within error and the ordering is noise. The minimum E_{ads} is still meaningful; which site achieves it is not.

Speed reference

Measured on this machine, MACE on CPU. Cost is roughly proportional to atom count times step count.

Knob	Effect on wall time
<code>--supercell 1,1</code> instead of <code>4,2</code>	About 8x faster per step (24 atoms instead of 192). Raises coverage to unphysical levels, so smoke tests only.
<code>--fmax 0.2</code> instead of <code>0.02</code>	Large. Fewer steps to converge; the single biggest lever after supercell.
<code>--max-steps 20</code>	Caps the worst case. Most relaxations converge near 90 steps.
<code>--max-sites 6</code> instead of uncapped	About 3.5x fewer adsorbate relaxations, and makes materials comparable.
<code>--protocol full_pipeline</code> instead of <code>both</code>	Roughly halves candidate cost.
<code>run_stage.py --from ...</code>	Skips every earlier stage entirely. Often the biggest saving of all.

Rough scale: one full material with O₂ at `--max-sites 6`, reference plus one candidate, `full_pipeline`, is about 52 relaxations and 4-5 hours. The seven-material family is about 31 hours. A smoke pass over all seven is about 20 minutes.